P1132R6
out_ptr - a scalable output pointer abstraction

Published Proposal, 2019-10-07

Authors:

JeanHeyd Meneide
Todor Buyukliev
Isabella Muerte

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:

C++23

Latest:

https://thephd.github.io/vendor/future_cxx/papers/d1132.html

Implementation:

https://github.com/ThePhD/out_ptr

Reply To:

JeanHeyd Meneide | @thephantomderp

Abstract

out_ptr and inout_ptr are abstractions to bring both C APIs and smart pointers back into the promised land by creating a temporary pointer-to-pointer that updates (using a reset call or semantically equivalent behavior) the smart pointer when it destructs.

Table of Contents

§ 1. Revision History

§ 1.1. Revision 6 - October 7th, 2019

— Library Evolution Working Group roundtrip NOT done: additional trait like std::ranges to make it easier to reject patently wrong arguments.

— Will join its sister paper, `std::retain_ptr` -- yay!

— Constructors are marked non-`noexcept` to allow for implementations to strengthen them as per their implementation strategies. This includes leaving it non-`noexcept` and forward-allocating `std::shared_ptr`'s control block in the constructor.

— Additional

§ 1.2. Revision 5 - July 10th, 2019

— Add back `void**` conversion after users voiced legitimate concerns about not being able to properly handle `std::unique_ptr<Base>` -> `out_ptr<Derived*>` -> `void**` functions, a common part of COM interfaces. This is explained in detail at the end of §3.5.1 Casting Support: easy void** support.

— Get benchmarks with both the unconditional noexcept and conditional noexcept code, particularly with non-`noexcept` `reset` calls (e.g. with `std::shared_ptr`). Results show it does not matter for the happy path, and therefore is being reverted.

— Prepare for a trip back to LEWG to answer the unconditional `noexcept` vs. conditional `noexcept`, favoring unconditional `noexcept`.

1.3. Revision 4 - June 17th, 2019

— Remove default `void**` conversion after discussion with LEWG and Boost members.

— Forwarded to LWG: just needs to be wording approved now!

1.4. Revision 3 - January 21st, 2019

— Add details discussing upcoming implementations in §4 Implementation Experience.

— Add section about implementation churn.

— Add details about flow control statements and temporary evaluation for §3.7 Footguns?.

— Wording is now relative to [n4820], the latest working draft of the C++ Standard.

1.5. Revision 2 - November 26th, 2018

— Add §3.7 Footguns? section.

— Discuss implementation experience in more detail.

— Add exception inspection and reasoning.

— Add additional design decision information about customization points.

1.6. Revision 1 - October 7th, 2018

— Add wording. Incorporate wording feedback. Eliminate CTAD design. Add a few more words about implementation experience.

1.7. Revision 0

— Initial release.

2. Motivation

> You're right that code shouldn't be using shared_ptr, I was trying to make it work with as little change as possible but after that and other more recent problems I'm finding a huge refactoring less and less avoidable. I'll make sure to turn everything into unique_ptr (there is no shared ownership anyways).
>
> Your out_ptr will still be massively helpful. — *King_DuckZ, September 25th, 2018*

This library automates the `.reset(...)` and -- sometimes additionally -- the `.release()` call for smart pointers when interfacing with `T**` output arguments.

Shared Code
From libavformat

```
#include <memory>
#include <avformat.h>

struct AVFormatContextDeleter {
                void operator() (AVFormatContext* c) noexcept const {
                        avformat_close_input(&c);
                        avformat_free_context(c);
                }
};
typedef std::unique_ptr<AVFormatContext, AVFormatContextDeleter> av_format_context_ptr;
// Signature from libavformat:
// int avformat_open_input(AVFormatContext **ps, const char *url, AVInputFormat *fmt, AVDictionary **optio
```

Current Code	With Prop

```
int main (int, char* argv[]) {
        av_format_context_ptr context(avformat_alloc_context());
        // ...
        // used, need to reopen
        AVFormatContext* raw_context = context.release();
        if (avformat_open_input(&raw_context,
                argv[0], nullptr, nullptr) != 0) {
                std::stringstream ss;
                ss << "ffmpeg_image_loader could not open file '"
                        << path << "'";
                throw FFmpegInputException(ss.str().c_str());
        }
        context.reset(raw_context);

        // ... off to the races !

        return 0;
}
```

```
int main (int, char* argv[]) {
        av_format_context_ptr conte
        // ...
        // used, need to reopen

        if (avformat_open_input(std
                argv[0], nullptr, n
                std::stringstream s
                ss << "ffmpeg_image
                        << argv[0]
                throw FFmpegInputEx
        }


        // ... off to the races!

        return 0;
}
```

We have very good tools for handling unique and shared resource semantics, alongside more coming with Intrusive Smart Pointers. Independently between several different companies, studios, and shops -- from VMWare and Microsoft to small game development startups -- a common type has been implemented. It has many names: `ptrptr`, `OutPtr`, `PtrToPtr`, `out_ptr`, WRL::ComPtrRef, a proposal on std-proposals and even unary operator& on CComPtr. It is universally focused on one task: making it so a smart pointer can be passed as a parameter to a function which uses an output pointer parameter in C API functions (e.g., `my_type**`).

This paper is a culmination of a private survey of types from the industry to propose a common, future-proof, high-performance `out_ptr` type that is easy to use. It makes interop with pointer types a little bit simpler and easier for everyone who has ever wanted something like `my_c_function( &my_unique );` to behave properly.

In short: it's a thing convertible to a `T**` that updates (with a reset call or semantically equivalent behavior) the smart pointer it is created with when it goes out of scope.

§ 3. Design Considerations

The core of `out_ptr`'s (and `inout_ptr`'s) design revolves around avoiding the mistakes of the past, preventing continual modification of new smart pointers and outside smart pointers's interfaces to perform the same task, and enabling some degree of performance efficiency without having to wrap every C API function.

§ 3.1. Synopsis

The function template's full specification is:

```
namespace std {
        template <class Pointer, class Smart, class... Args>
        out_ptr_t<Smart, Pointer, Args...>
        out_ptr(Smart& s, Args&&... args) noexcept;

        template <class Smart, class... Args>
        out_ptr_t<Smart, POINTER_OF(Smart), Args...>
        out_ptr(Smart& s, Args&&... args) noexcept;

        template <class Pointer, class Smart, class... Args>
        inout_ptr_t<Smart, Pointer, Args...>
        inout_ptr(Smart& s, Args&&... args) noexcept;

        template <class Smart, class... Args>
        inout_ptr_t<Smart, POINTER_OF(Smart), Args...>
        inout_ptr(Smart& s, Args&&... args) noexcept;
}
```

Where `POINTER_OF` is the `::pointer` type, then the `::element_type*` type, then `typename std::pointer_traits<Smart>::element_type*` type in that order. The return type `out_ptr_t` and its sister type `inout_ptr_t` are templated types and must at-minimum have the following:

```
template <class Smart, class Pointer, class... Args>
class out_ptr_t {
public:
        out_ptr_t(Smart&, Args...);
        ~out_ptr_t () noexcept;
        operator Pointer* () noexcept const;
        operator void** () noexcept const;
        // ^ only if Pointer != void*
};

template <class Smart, class Pointer, class... Args>
class inout_ptr_t {
public:
        inout_ptr_t(Smart&, Args...);
        ~inout_ptr_t () noexcept;
        operator Pointer* () noexcept const;
        operator void** () noexcept const;
        // ^ only if Pointer != void*
};
```

We specify "at minimum" because we expect users to override this type for their own shared, unique, handle-alike, reference-counting, and etc. smart pointers. The destructor of `~out_ptr_t()` calls `.reset()` on the stored smart pointer of type `Smart` with the stored pointer of type `Pointer` and arguments stored as `Args...`. `~inout_ptr_t()` does the same, but with the additional caveat that the constructor for `inout_ptr_t(Smart&, Args&&...)` also calls `.release()`, so that a `reset` doesn't double-delete a pointer that the expected re-allocating API used with `inout_ptr` already handles.

We chose this extension point because the other options (ADL extension, friend ADL extension) have proven to not be very feasible in the long run of maintainability. While we are wary that users open up namespace `std` we also recognize that it is essentially the best way that someone can extend this type to pointers and handles that are _not_ part of the standard. If this only works with standard types -- and only standard types that are explicitly sanctioned -- then this type is almost certainly not worth it. See §3.8 Extension Points for more details.

§ 3.2. Overview

`out_ptr`/`inout_ptr` are free functions meant to be used for C APIs:

```
error_num c_api_create_handle(int seed_value, int** p_handle);
error_num c_api_re_create_handle(int seed_value, int** p_handle);
void c_api_delete_handle(int* handle);

struct resource_deleter {
        void operator()( int* handle ) {
                c_api_delete_handle(handle);
        }
};
```

Given a smart pointer, it can be used like so:

```
std::unique_ptr<int, resource_deleter> resource(nullptr);
error_num err = c_api_create_handle(
        24, std::out_ptr(resource)
);
if (err == C_API_ERROR_CONDITION) {
        // handle errors
}
// resource.get() the out-value from the C API function
```

Or, in the re-create (reallocation) case:

```
std::unique_ptr<int, resource_deleter> resource(nullptr);
error_num err = c_api_re_create_handle(
        24, std::inout_ptr(resource)
);
if (err == C_API_ERROR_CONDITION) {
        // handle errors
}
// resource.get() the out-value from the C API function
```

§ 3.3. Safety

This implementation uses a pack of `...Args` in the signature of `out_ptr` to allow it to be used with other types whose `.reset()` functions may require more than just the pointer value to form a valid and proper smart pointer. This is the case with `std::shared_ptr` and `boost::shared_ptr`:

```
std::shared_ptr<int> resource(nullptr);
error_num err = c_api_create_handle(
        24, std::out_ptr(resource, resource_deleter{})
);
if (err == C_API_ERROR_CONDITION) {
        // handle errors
}
// resource.get() the out-value from
// the C API function
```

Additional arguments past the smart pointer stored in `out_ptr`'s return type will perfectly forward these to whatever `.reset()` or equivalent implementation requires them. If the underlying pointer does not require such things, it may be ignored or discarded (optionally, with a compiler error using a static assert that the argument will be ignored for the given type of smart pointer).

Of importance here is to note that `std::shared_ptr` can and will overwrite any custom deleter present when called with just `.reset(some_pointer);`. Therefore, we make it a compiler error to not pass in a second argument when using `std::shared_ptr` without a deleter:

```
std::shared_ptr<int> resource(nullptr);
error_num err = c_api_create_handle(
        42, std::out_ptr(resource)
); // ERROR: deleter was changed
   // to an equivalent of
   // std::default_delete!
```

It is likely the intent of the programmer to also pass the fictional `c_api_delete_handle` function to this: the above constraint allows us to avoid such programmer mistakes.

§ 3.4. Safety: Exceptions

This is two-fold. First, by placing the `.reset()` call into the destructor of `out_ptr`/`inout_ptr`, we can guarantee safety that trivial code does not have. For example, consider this abstracted form of the production code shown in the Tony Table:

```
std::unique_ptr<int> num(new int());
// use, then have to prepare for some
// c_api call
int* raw_num = num.release();
if (my_c_api_call(&raw_num) != 0) {
        // leak if the c api call does nothing!!
        throw std::runtime_error("leaking memory!");
}
num.reset(raw_num);
```

If the user used `std::inout_ptr`, the value would be guaranteed to put back into the unique pointer, and then subsequently destroyed as the stack continued to be unwound.

Secondly, the destructor for `out_ptr` calls to `.reset()`. The only case where this is questionable is with `std::shared_ptr`: the creation of the passed-in deleter might throw, and thusly the call cannot be `noexcept`. This means that the destructor might throw if `std::shared_ptr's .reset()` throws: in this case, `std::terminate` would be called.

In practice, this has not been observed (or reported). Still, the C++ standard makes guarantees for code even if the situation is never encountered in the real world: for this, it is much more palatable to go with the status quo and stick to the policies established by LEWG and LWG regarding throwing until such a policy is overturned.

Additionally, functions which have a non-`noexcept` reset function are guaranteed to delete the resource before releasing the exception, as is specified in e.g. [util.smartptr.shared.const]. This means that it is the responsibility of the pointer to release any resources once their `reset(...)` (or equivalent) function is successfully called.

Finally, the original design had an unconditionally `noexcept` constructor. It has been suggested that the constructor be non-noexcept, and to allow an implementation to prevent throwing in the destructor by forward-allocating any necessary internal storage for the `reset` operation (for example, the control block of a `shared_ptr`) in the constructor. This would prevent termination from happening in the destructor when a common non-noexcept `reset` call is invoked. We leave answering this question to the discretion of LEWG.

§ 3.5. Casting Support

There are also many APIs (COM-style APIs, base-class handle APIs, type-erasure APIs) where the initialization requires that the type passed to the function is of some fundamental (`void**`) or base type that does not reflect what is stored exactly in the pointer. Therefore, it is necessary to sometimes specify what the underlying type `out_ptr` uses is stored as.

It is also important to note that going in the *opposite* direction is also highly desirable, especially in the case of doing API-hiding behind an e.g. `void*` implementation. `out_ptr` supports both scenarios with an optional template argument to the function call.

§ 3.5.1. Casting Support: easy `void**` support

Consider this DirectX Graphics Infrastructure Interface (DXGI) function on `IDXGIFactory6`:

```
HRESULT EnumAdapterByGpuPreference(
        UINT Adapter,
        DXGI_GPU_PREFERENCE GpuPreference,
        REFIID riid,
        void** ppvAdapter
);
```

Using `out_ptr`, it becomes trivial to interface with it using an exemplary `std::unique_ptr<IDXGIAdapter, ComDeleter> adapter`:

```
HRESULT result = dxgi_factory.
EnumAdapterByGpuPreference(0,
        DXGI_GPU_PREFERENCE_MINIMUM_POWER,
        IID_IDXGIAdapter,
        std::out_ptr(adapter)
);
if (FAILED(result)) {
        // handle errors
}
// adapter.get() contains strongly-typed pointer
```

No manual casting, `.release()` fiddling, or `.reset()` is required: the returned type from `out_ptr` handles that. This is because the `out_ptr_t` and `inout_ptr_t` types have conversion operations to not only the detected `::pointer` or `::element_type*` of the smart pointer, but a `reinterpret_cast` conversion to `void*` as well. While the size of `void*` is not required by the C++ standard to be the same as the size of any other types pointer (except const/volatile qualified `char*`), most C APIs that use this technique have already sanctioned the conversion from whatever type the API works with to `void*` and, subsequently, `void**`.

This idiom is also useful for the `QueryInterface` base function for COM's `IUnknown`, and for Vulkan's `vkMapMemory`.

Note that the implicit `void**` conversion is important for more than just easy interaction with Windows or COM APIs: it is commonplace to store a pointer to a base class rather than a derived one. For example:

```
std::unique_ptr<ID3D11Device> g_d11_device;

int main () {
        init_global_device();

        std::unique_ptr<IUnknown> dxgi_device;
        IDXGIDevice * pDXGIDevice;
        HRESULT hr = g_d11_device->QueryInterface(
                __uuidof(IDXGIDevice),
                std::out_ptr<void*>(pDXGIDevice)); // !!
        if (FAILED(hr)) {
                // ...
        }
        // ...
        return 0;
}
```

The above code is actually wrong. `IUnknown` cast to `void*`, then passed as `void**` to the function, creates a bad pointer because the `void**` inside of the function call is cast directly to `IDXGIDevice**` and dereferenced. This is incorrect because the original pointer is `IUnknown`, and so member variables and the object's virtual table will be completely skewed and out of place. The correct way is to write use `QueryInterface` as so:

```
std::unique_ptr<ID3D11Device> g_d11_device;

int main () {
        init_global_device();

        std::unique_ptr<IUnknown> dxgi_device;
        IDXGIDevice * pDXGIDevice;
        HRESULT hr = g_d11_device->QueryInterface(
                __uuidof(IDXGIDevice),
                std::out_ptr<IDXGIDevice*>(pDXGIDevice));
        if (FAILED(hr)) {
                // ...
        }
        // ...
        return 0;
}
```

This properly up-casts to the derived type, and then decays to the desired `void**` and works properly. This problem was discovered when a version of `out_ptr` not containing the `void**` implicit conversion was shipped to customers who depended on this feature to do the correct base -> derived -> `void**` conversion. Therefore, the original design was restored to keep it up to date.

§ 3.5.2. Casting Support: to arbitrary `T`

In many cases, there is a typical C structure or similar that C++ users are sanctioned to derive and extend with their own data, with the promise that as long as the pointed passed to the function has a base class or matching type. It also happens that someone needs to cast from a type-erased `void*` to a more-derived type. There are also cases where the type stored in `std::unique_ptr<T, Deleter>` uses `Deleter` to override the `::pointer` type, making `std::unique_ptr` store the (fat, offset) `::pointer` that is convertible to `T*`.

For example, one technique detailed by a graphics develop helped them make an agnostic `graphics_handle` type: a type-erased pointer for DirectX or a regular integer for OpenGL. This requires casting from a chunk of type-erased storage to a more concrete `ID3D11Texture*` or similar. Allowing for `out_ptr` to work on that level was critical for its usage in these cases.

It is imperative that the user be allowed to specify a casting parameter that the `out_ptr_t`/`inout_ptr_t`, and that is done by simply adding a type when calling the desired function. Consider a specialized `std::unique_ptr<int, fd_deleter>` where `::pointer` is a typedef to a special `fd` type:

```
struct fd {
        int handle;

        fd()
        : fd(nullptr) {}
        fd(std::nullptr_t)
        : handle(static_cast<intptr_t>(-1)) {}
        fd(FILE* f)
#ifdef _WIN32
        : handle(f ? _fileno(f) : static_cast<intptr_t>(-1)){
#else
        : handle(f ? fileno(f) : static_cast<intptr_t>(-1)) {
#endif // Windows
        }

        explicit operator bool() const;

        bool operator==(std::nullptr_t) const;
        bool operator!=(std::nullptr_t) const;
        bool operator==(const fd& fd) const;
        bool operator!=(const fd& fd) const;
};

struct fd_deleter {
        using pointer = fd;
        void operator()(fd des) const;
};
```

Casting in this case is cumbersome and often error-prone to do properly when interfacing with C or C++ standard library facilities. It becomes trivial with `std::out_ptr`:

```
std::unique_ptr<int, fd_deleter> my_unique_fd;
auto err = fopen_s( std::out_ptr<FILE*>(my_unique_fd), "prod.csv", "rb" );
// check err, then work with raw fd
```

This is an example of a codebase which works primarily off of file descriptors, but wants to interop with the standard C and C++ libraries. The cast here is valid and properly opens the file, while the `fd` type handles converting in and out of the type safely and seamlessly, without going through extra effort or having to interact more closely with the POSIX API. This makes it easy to perform interop with a "high-level" or "convertible" type, while still working with the desired "low-level" or "native" type.

This also demonstrates `out_ptr`'s ability to work with offset/fat/not-quite-exactly pointers, which are allowed by `std::unique_ptr` and the upcoming std::retain_ptr.

The full example code for Windows and *Nix platforms is available as a compile-able example.

§ 3.6. Reallocation Support

In some cases, a function given a valid handle/pointer will delete that pointer on your behalf before performing an allocation in the same pointer. In these cases, just `.reset()` is entirely redundant and dangerous because it will delete a pointer that it does not own. Therefore, there is a second abstraction called `inout_ptr`, so aptly named because it is both an input (to be deleted) and an output (to be allocated post-delete). `inout_ptr`'s semantics are exactly like `out_ptr`'s, just with the additional requirement that it calls `.release()` on the smart pointer upon constructing the temporary `inout_ptr_t`.

This can be heavily optimized in the case of `unique_ptr`, but to do so from the outside requires Undefined Behavior or modification of the standard library. See §5.2 For std::inout_ptr for further explication.

§ 3.7. Footguns?

As far as we know and have designed this specification, `std::out_ptr` and `std::inout_ptr` have no hidden or easy-to-access footguns for its intended usage. Originally, `std::out_ptr` was going to potentially include a runtime parameter to encapsulate the behavior of `std::inout_ptr`: however, it was deemed much better design to separate the two out into separate functions. This also matched VMWare's implementation experience with the type and generated far superior code. It also made it easier to know when to pick `out_ptr` versus `inout_ptr`: one is for regular allocations that just create something new, the other is for the case when you need to reallocate into the pointer and thusly can save some instructions.

Furthermore, all examples of `out_ptr`/`inout_ptr` include usage as a temporary to a function call. Let us assume someone wanted to get sufficiently clever:

```
std::unique_ptr<int> u_ptr;
auto op = std::out_ptr(u_ptr);
int err = c_function_call(op);
if (err != 0) {
        throw std::runtime_error()
}
```

This still behaves the same: but, `.reset()` will be called before the `unique_ptr` goes out of scope. Unless the user performs extraordinary gymnastics to circumvent the typical lifetime of the factory-generated `out_ptr`, there are no footguns in regular and general usage.

The only other place where someone could be sufficiently clever is with a function call _and_ a flow control statement. For example, an `if` statement that initializes something and also tests the smart pointer in that same `if` statement will extend lifetimes in a very poor order:

```
std::unique_ptr<foo_handle, foo_deleter> my_unique(nullptr);

if (get_some_pointer(std::out_ptr(my_unique)); my_unique)) {
        std::cout << "yay" << std::endl;
}
else {
        std::cout << "oh no" << std::endl;
}
```

This happens whether the expression is chained with multiple comma/conditional expressions or if someone uses the new flow-control initializer statements. This is an unfortunately holdover of how temporaries are treated, and rather being fixed with flow control initializer statements the same quirky rules for the old `if` were carried over.

This was pointed out as strange, but we feel this is not much of a blocker for this proposal. All RAII-based, action-on-destroy resources suffer from this problem: it is neither a new nor novel problem. One does not use a `std::lock_guard` in similar fashion to the snippet above; neither should `std::out_ptr` be used to that effect. Even Microsoft's Raymond Chen takes issues with temporary destructors and lifetimes in `if` statements and similar, but takes that problem up with C++ in general, not the abstraction presented in the write up or here.

In most cases, when working with C APIs, there are error numbers to check which are infinitely more reliable than testing the pointer. APIs would do better to switch to such a method, rather than relying on testing for the null state on a pointer. Which is another way of saying, the following code is still just fine:

```
std::unique_ptr<foo_handle, foo_deleter> my_unique(nullptr);

if (auto err = get_some_pointer(std::out_ptr(my_unique)); err == 0)) {
        std::cout << "yay" << std::endl;
}
else {
        std::cout << "oh no" << std::endl;
}
```

3.8. Extension Points

A number of extension points were considered for this proposal. We have purposefully selected the ability to specialize the class template because it is the most flexible approach that allows library authors outside of the `std::` namespace customize their types to work properly. This proposal rose primarily out of seeing many _different_ kinds of smart pointers handled in many codebases, from hobby to industry, that are currently not covered (and likely not to be covered in the near future) by the standard. Therefore, an extension mechanism that is available to library authors and users seems to be the most efficient.

It is also important that we limit the surface area in which the user can harm themselves and their users. ADL, for example, can cause supreme danger because the overloads of `std::out_ptr` and `std::inout_ptr` are variadic forwarding templates which handle when a user might want to pass additional arguments to `offset_ptr` or similar. This can be quite dangerous as it is ripe territory for ambiguities.

Class template specialization requires exactly matching arguments and does not suffer from potential convertibility in which other solutions might pick wrong overloads or select the wrong extension call because of mixed-namespace arguments. It also prevents build breaks from being introduced in subtle and hard-to-catch ways. It is also much less likely for someone to try to apply `std::enable_if_t` or Concept constraints on their template class specializations to resolve ambiguities because of the exact-matching feature, as opposed to functions where partial and full specialization are hazardous and error-prone to get right.

Below are catalogued some explored and **ultimately rejected** customization points.

3.8.1. Rejected: just adding `get_ref` to related non-shared pointers

This solution seeks to resolve performance problems and reseating issues by having `std::unique_ptr` add a `T*& get_ref();` function on itself that an `inout_ptr` solution or C function user might take advantage of. The problem is this breaks encapsulation over its knee and destroys and integrity the pointer value has from `unique_ptr`'s invariant. Additionally, it means that all libraries have to provide a function on their types that they currently do not provide (and for very good reason). While tempting as low-hanging fruit, this is an extraordinary example of a simple design which has far-reaching, poor consequences.

3.8.2. Rejected: adding `&operator` to this type

This is the same sin committed by Microsoft with `CComPtr` that ushered in the age of `std::addressof` with all due experience for Windows users. While proposed a few times throughout history (including in the early incubation tank of std-proposals), this is not a mistake the community should make twice.

3.8.3. Rejected: unrestricted ADL

`std::swap` works out fairly nicely as an extension point. Coming up with a fairly expansive name that is not as common as `swap` and designating that to be the ADL extension point could be worth doing. Also creating callable Customization Point Objects that `using std::the_func` before calling `the_func` in an unqualified manner is similar to the design decision ranges made.

Unfortunately, ADL is also entirely unconstrained once opened up in this manner. It takes careful programming and perhaps a bit of SFINAE to ensure there are no collisions, especially in the "base cases" users might want to specialize for. This can lead to brittle code that breaks when we ship updates to the desired ADL extension point, or users that under or over-constrain their version of the function. It exposes too much surface area for the programmer to load not a footgun but a landmine that either their future selves, coworkers, or left-behind future colleagues might get lost on.

It is a considerable contender but for the above reasons -- especially since `out_ptr`/`inout_ptr` need to have unconstrained variadic arguments to pass additional extra arguments to pointers like `boost::`/`std::shared_ptr` or `boost::intrusive_ptr` or the upcoming `std::retain_ptr` -- it is rejected.

3.8.4. Rejected: restricted ADL using in-class friend functions

At first, this idea is tempting. It is used in abseil to e.g. provide hash customization and allows the implementer to access the internals of the pointer they are adding it to. Having a `static friend` function seems to cover the biggest risks (asides from template footguns in the previous section). In a world where building from source and owning your dependencies is ideal, or being able to freeze versions at will and edit code that you know is abandon-ware, this seems like the ideal solution that covers most use cases. It also seems to prevent the more naughty use cases of ADL.

Unfortunately, this requires opt-in from every author of a library type. This means that either you fork the library to your own version and patch it, maintain a patch in the case of an author who does not deem you adding that extension point useful, or just own the library and stay up to date. While feasible for large teams that have bandwidth to spend on this problem, this is problematic for smaller teams and hobby developers. It is a good way to do extension, but it is a novel idea and only tested within a few libraries. There are also issues of legality when performing modification to headers and compiled code directly to support this idiom: `out_ptr` as currently designed does not fall prey to such problems.

Already, users have sent me tweets and e-mails about extending this for their own types that they do not own. It would defeat the purpose of this type to require explicit opt-in.

3.8.5. Potential Future: Traits type customization

Some degree of success has been had with a `out_ptr_traits` and a `inout_ptr_traits` customization point. It reduced the amount of code necessary to write over the original structure-based version, even if it came at the loss of some control of the underlying class and its temporary storage. If stack space is at a premium, the traits specialization when used to optimize `std::unique_ptr`-like changes is actually mildly inferior, but only in the case of taking up an extra pointer or two of stack space. This is likely negligible for the vast majority of use cases.

We propose that the struct specialization stays because it gives the user the most control for truly unique circumstances. Users should not be overriding the base defaults often except in the case to severely rework the internals. It is also possible to implement the traits type customization as a future extension: this has already been deployed in the version proposed to Boost as well as the standalone C++ version vended to companies and individuals around the world. It does not have the same degree of usage experience as the struct specialization, however, so this paper will not propose it until it can be seen as successfully replacing the old usages without problem.

4. Implementation Experience

This library has been brewed at many companies in their private implementations, and implementations in the wild are scattered throughout code bases with no unifying type. As noted in §2 Motivation, Microsoft has implemented this in `WRL::ComPtrRef`. Its earlier iteration -- `CComPtr` -- simply overrode `operator&`. We assume they prefer the former after having forced the need with `CComPtr` for `std::addressof`. the WRL is a public library used in thousands of applications, and has an interface similar to the proposed `std::out_ptr`/`std::inout_ptr`.

VMWare has a type that much more closely matches the specification in this paper, titled `Vtl::OutPtr`. The primary author of this paper wrote and used `out_ptr` for over 5 years in their code base working primarily with graphics APIs such as DirectX and OpenGL, and more recently Vulkan. They have also seen a similar abstraction in the places they have interned at.

Similarly, Adobe's Chromium project has its own version of out_ptr.

The primary author of [p0468] in pre-r0 days also implemented an overloaded `operator&` to handle interfacing with C APIs, but was quickly talked out of actually proposing it when doing the proposal. That author has joined in on this paper to continue to voice the need to make it easier to work with C APIs without having to wrap the function.

Given that many companies, studios and individuals have all invented the same type independently of one another, we believe this is a strong indicator of agreement on an existing practice that should see a proposal to the standard.

A full implementation with UB and friendly optimizations is available in the repository. The type has been privately used in many projects over the last four years, and this public implementation is already seeing use at companies today. It has been particularly helpful with many COM APIs, and the re-allocation support in `inout_ptr` has been useful for FFMPEG's functions which feature reallocation support in their functions (e.g., `avformat_open_input`).

A version of this library was reviewed for Boost and rejected, but with encouragement to come back once the minor requested changes were done. However, re-proposing to Boost is not a good idea, since they want a version without optimizations and that version would lack the ability to pry on `std::shared_ptr` to pre-allocate in the constructor to keep the destructor `noexcept`. This makes it more of a primary candidate for the Standard than Boost, since it requires help directly from standard library implementers (or UB on the part of an outside implementation, which my 3rd party implementation engages in but Boost does not want at all).

§ 4.1. Why Not Wrap It?

A common point raised while using this abstraction is to simply "wrap the target function". We believe this to be a non-starter in many cases: there are thousands of C API functions and even the most dedicated of tools have trouble producing lean wrappers around them. This tends to work for one-off functions, but suffers scalability problems very quickly.

Templated intermediate wrapper functions which take a function, perfectly forwards arguments, and attempts to generate e.g. a `unique_ptr` for the first argument and contain the boiler plate within itself also causes problems. Asides from the (perhaps minor) concern that such a wrapping function disrupts any auto-completion or tooling, the issue arises that C libraries -- even within themselves -- do not agree on where to place the `some_c_type**` parameter and detecting it properly to write a generic function to automagically do it is hard. Even within the C standard library, some functions have output parameters in the beginning and others have it at the end. The disparity grows when users pick up libraries outside the standard.

§ 4.2. Conditionally `noexcept` destructor?

There is no indication whatsoever that conditional noexcept specification has any performance benefits in the happy path, as shown by the §5.3 For std::shared_ptr and various cases section. That makes the question of `noexcept(computed-expression)` vs. `noexcept(true)` strictly a matter of correctness, and not one of leaving no additional room beneath this library for a better C or hand-written alternative.

Research into the Standard Library and scanning of the latest working draft, [n4820], there is only a handful of destructors that could have potentially been marked throwing. However, due to the guidance of [res.on.exception.handling], all of them have been marked as `noexcept`. Even the one destructor that logically could have thrown is marked explicitly `noexcept`, and it makes sure the "exception is caught but not rethrown" ([filebuf.cons], clause 5). While nobody is going to argue that `<iostream>`s and anything related to it is a paragon of good design, there is clear precedent in the Standard Library not to throw in destructors no matter what.

Furthermore, there are plenty of things in the standard which error and should throw to communicate that error, but errors are deliberately not mentioned or ignored for the sake of destructors and usage idioms in the standard library. Consider the class of mutex types in the standard, including `std::mutex`: their unlock operations on all platforms can signal Out of Resource-style errors (`EAGAIN` for resource exhaustion pthreads mutices, for example). Absolutely no platforms throw an error for this, including libstdc++ which simply ignores the error even after acknowledging it in comments and libc++ which only has an assertion tanking in an assert-enabled build of libc++, otherwise nothing. While this is not "as bad" as `basic_filebuf`, it's morally equivalent to just not bothering with the error: `std::lock_guard` could trigger any of these errors, and absolutely nothing will inform the programmer that something exploded, which is arguably *worse* than just letting the exception hit the `noexcept(true)` barrier of a standard destructor and tank the program because an important invariant has been violated.

In short, given the long policy and history of `noexcept`-marked destructors even in the face of errors, the lack of performance differentiation for the happy path with or without `noexcept` markings, and the clear guidance in both Specification and Implementation to drop out-of-resource errors on the floor, `out_ptr_t` and `inout_ptr_t` will both remain unconditionally `noexcept` to be consistent with C++ in letter, implementation and spirit.

§ 5. Performance

Many C programmers in our various engineering shops and companies have taken note that manually re-initializing a `unique_ptr` when internally the pointer value is already present has a measurable performance impact.

Teams eager to squeeze out performance realize they can only do this by relying on type-punning shenanigans to extract the actual value out of `unique_ptr`: this is expressly undefined behavior. However, if an implementation of `out_ptr` could

be friended or shipped by the standard library, it can be implemented without performance penalty.

Below are some graphs indicating the performance metrics of the code. 5 categories were measured:

— "c_code": handwritten C code, which does not use this idiom

— "clever": uses UB to alias the pointer value stored in `std::unique_ptr`

— "friendly": modifies VC++'s, libc++'s, and libstdc++'s `std::unique_ptr`s to allow the implementation to friend the `out_ptr` implementation, to access the internals without UB

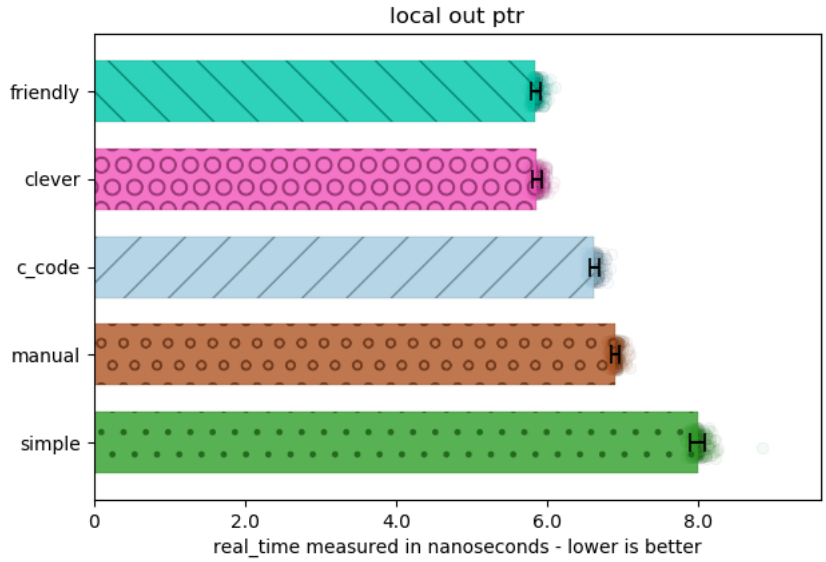
— "manual": does the work by-hand using reset/release from a `std::unique_ptr`

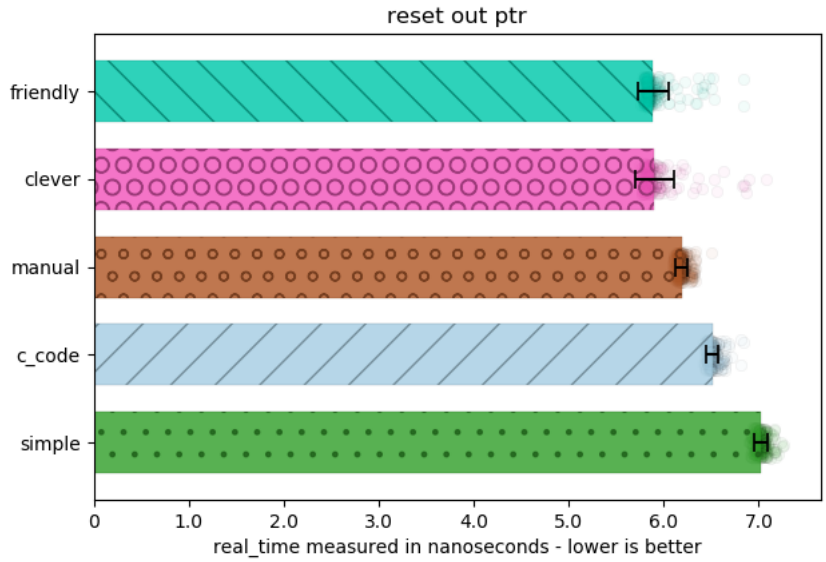
— "simple": a `out_ptr` implementation that naively resets

The full JSON data for these benchmarks is available in the repository, as well as all of the code necessary to run the benchmarks across all platforms with a simple CMake build system.

§ 5.1. For `std::out_ptr`

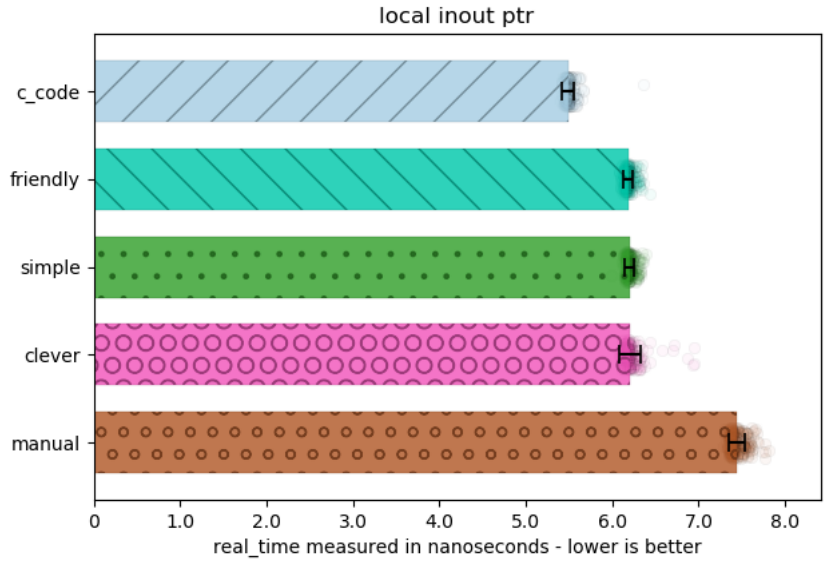
You can observe two graphs for two common `unique_ptr` usage scenarios, which are using the pointer locally and discarding it ("local"), and resetting a pre-existing pointer ("reset") for just an output pointer:

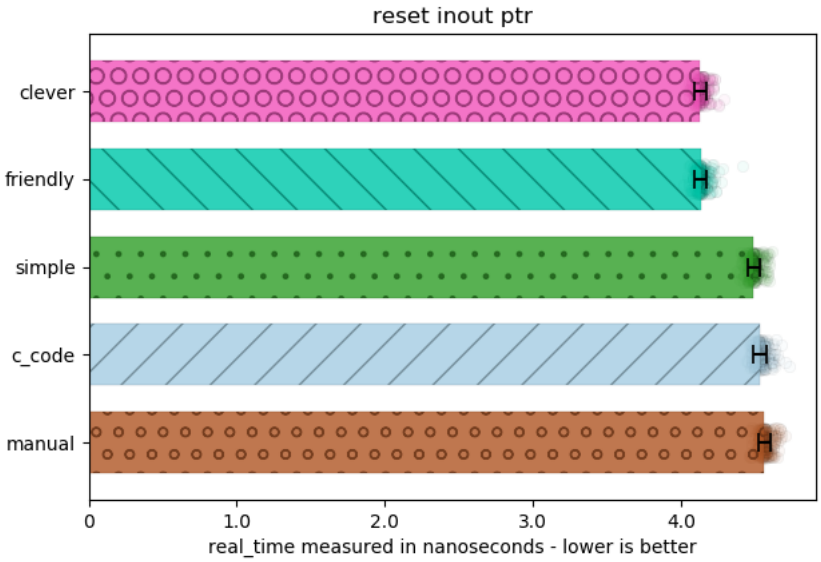




§ 5.2. For `std::inout_ptr`

The speed increase here is even more dramatic: reseating the pointer through `.release()` and `.reset()` is much more expensive than simply aliasing a `std::unique_ptr` directly. Places such as VMWare have to perform Undefined Behavior to get this level of performance with `inout_ptr`: it would be much more prudent to allow both standard library vendors and users to be able to achieve this performance without hacks, tricks, and other I-promise-it-works-I-swear pledges.

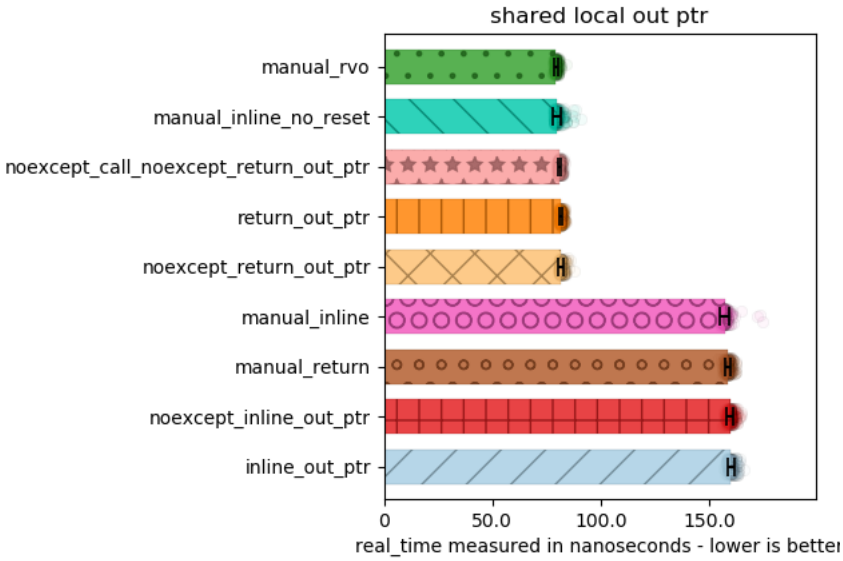


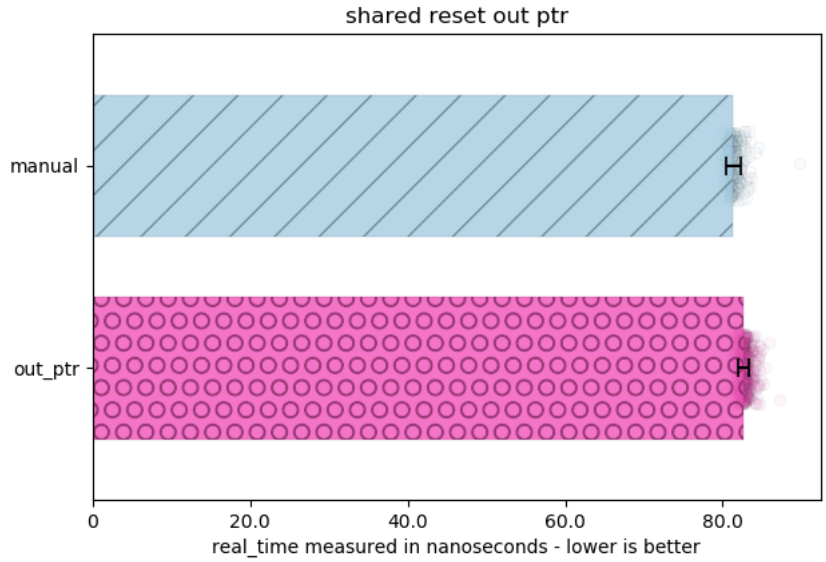


§ 5.3. For `std::shared_ptr` and various cases

Here, we examine the impact of `noexcept` and various styles of initializing a `std::shared_ptr` with a value. This is mostly informational and educational: due to the asynchronous, shared ownership semantics of `std::shared_ptr` there is little that can be done in the optimization space. Note that this is not the case for types like `boost::local_shared_ptr` and the upcoming `std::retain_ptr`, where inline and synchronous execution are part of the type's semantic contract with the user by default. This allows someone to directly reseat the resource within, depending on things like tracked reference count or whether it is conceptually an `out_ptr` operation or an `inout_ptr` operation.

Note that `noexcept`-ness does not change the performance of the happy path in any of these scenarios, regardless of whether a conditional `noexcept` or unconditionally `noexcept`:





§ 6. Bikeshed

As with every proposal, naming, conventions and other tidbits not related to implementation are important. This section is for pinning down all the little details to make it suitable for the standard.

§ 6.1. Alternative Specification

The authors of this proposal know of two ways to specify this proposal's goals.

The authors have settled on the approach in §3.1 Synopsis. We believe this is the most robust and easiest to use: singular names tend to be easier to teach and use for both programmers and tools. We discuss the older techniques to uphold thorough discussion and inspection of the solution space.

The first way is to specify both functions `out_ptr` and `inout_ptr` as factories, and then have their types named differently, such as `std::out_ptr_t` and `std::inout_ptr_t`. The factory functions and their implementation will be fixed in place, and users would be able to (partially) specialize and customize `std::out_ptr_t` and `std::inout_ptr_t` for types external to the stdlib for maximum performance tweaking and interop with types like `boost::shared_ptr`, `my_lib::local_shared_ptr`, and others. This is the direction this proposal takes.

The second way is to specify the class names to be `std::out_ptr` / `std::inout_ptr`, and then used Template Argument Deduction for Class Templates from C++17 to give a function-like appearance to their usage. Users can still specialize for types external to the standard library. This approach is more Modern C++-like, but contains a caveat.

Part of this specification is that you can specify the stored pointer for the underlying implementation of `out_ptr` as shown in §3.5 Casting Support. Template Argument Deduction for Class Templates does not allow partial specialization (and for good reason, see the interesting example of `std::tuple<int, int>{1, 2, 3}`). The "Deduction Guides" (or CTAD) approach would accommodate §3.5 Casting Support using functions with a more explicit names, such as `out_ptr_cast<void*>( ... );` and `inout_ptr_cast<void*>( ... );`. This route was deemed less overall palatable.

§ 6.2. Naming

Naming is hard, and therefore we provide a few names to duke it out in the Bikeshed Arena:

For the `out_ptr` part:

— out_ptr

— c_ptr

— c_out_ptr

— out_c_ptr

— alloc_c_ptr

— out_smart

— ptrptr

— ptr_to_ptr

— ptr_to_smart

— ptr_ref

For the `inout_ptr` part:

— inout_ptr

— c_in_ptr

— c_inout_ptr

— inout_c_ptr

— realloc_c_ptr

— inout_smart,

— realloc_ptr_to_ptr

— realloc_ptr_to_smart

— realloc_ptr_ref

As a pairing, `out_ptr` and `inout_ptr` are the most cromulent and descriptive in the authors' opinions. The type names would follow suit as `out_ptr_t` and `inout_ptr_t`. However, there is an argument for having a name that more appropriately captures the purpose of these abstractions. Therefore, `c_out_ptr` and `c_inout_ptr` would be even better, and the shortest would be `c_ptr` and `c_in_ptr`.

§ 7. Proposed Changes

The following wording is for the Library section, relative to [n4820]. This feature will go in the `<memory>` header, and is added to [**utilities.smartptr**] §20.11, at the end as subsection 9.

§ 7.1. Proposed Feature Test Macro and Header

This should be available with the rest of the smart pointers, and thusly be included by simply including `<memory>`. If there is a desire for more fine-grained control, then we recommend the header `<out_ptr>` (subject to change based on bikeshed painting above). There has been some expressed desire for wanting to provide more fine-grained control of what entities the standard library produces when including headers: this paper does not explicitly propose adding such headers or doing such work, merely making a recommendation if this direction is desired by WG21.

The proposed feature test macro for this is `__cpp_lib_out_ptr`. The exposure of `__cpp_lib_out_ptr` denotes the existence of both `inout_ptr` and `out_ptr`, as well as its customization points `out_ptr_t` and `inout_ptr_t`.

§ 7.2. Intent

The intent of this wording is to allow implementers the freedom to implement the return type from `out_ptr` as they so choose, so long as the following criteria is met:

— the return type is of the name `inout_ptr_t`/`out_ptr_t` and is a template with 3 template parameters;

— the destructor of `inout_ptr_t`/`out_ptr_t` properly re-seats the pointer owned/stored by whatever smart/fancy pointer is passed as the first argument to `out_ptr`;

— the proper implicit conversion operators are added to the type,

— the standard library implementation itself does not specialize `inout_ptr_t`/`out_ptr_t`'s templates, either fully or partially, so that the user can override the behavior of these templates as they so choose;

— `std::shared_ptr` used with the `out_ptr` or `inout_ptr` functions will produce a diagnostic if it is called without a second argument meant to be passed as the deleter to a `.reset()` call; and

— `std::shared_ptr` used with `inout_ptr_t` will always result in a diagnostic because it is impossible to completely release the resource from its shared ownership.

The goals of the wording are to not restrict implementation strategies (e.g., a `friend` implementation as benchmarked above for `unique_ptr`, or maybe a UB/IB implementation as also documented above). It is also explicitly meant to error for smart pointers whose `.reset()` call may reset the stored deleter (á la `boost::shared_ptr`/`std::shared_ptr`) and to catch programmer errors.

§ 7.3. Proposed Wording

Append to §17.3.1 General [**support.limits.general**]'s **Table 35** one additional entry:

Macro name	Value	Header(s)
`__cpp_lib_out_ptr`	`201907L`	`<memory>`

Modify §20.10.1 In general [**memory.general**] as follows:

> [1] The header defines several types and function templates that describe properties of pointers and pointer-like types, manage memory for containers and other template types, destroy objects, and construct multiple objects in uninitialized memory buffers (20.10.3–19.10.11). The header also defines the templates unique_ptr, shared_ptr, weak_ptr, `out_ptr_t`, `inout_ptr_t`, and various function templates that operate on objects of these types (20.11).

Add §20.10.2 Definitions [**memory.defns**] as follows:

> [1] Definition: Let `POINTER_OF_OR(T, U)` denote a type that is:
>
> — `T::pointer` if the qualified-id `T::pointer` is valid and denotes a type, or
>
> — otherwise, `typename T::element_type*` if the qualified-id `T::element_type` is valid and denotes a type,
>
> — otherwise, `U`.
>
> [2] Definition: Let `POINTER_OF(T)` denote a type that is defined as `POINTER_OF_OR(T, typename std::pointer_traits<T>::element_type*)`.

Add to §20.10.3 (previously §20.10.2) Header `<memory>` synopsis [**memory.syn**] the `out_ptr`, `inout_ptr`, `out_ptr_t` and `inout_ptr_t` functions and types:

```
// 20.11.9, out_ptr_t
template <class Smart, class Pointer, class... Args>
  class out_ptr_t;

// 20.11.10, out_ptr
template <class Pointer, class Smart, class... Args>
  out_ptr_t<Smart, Pointer, Args...> out_ptr(Smart& s, Args&&... args);

template <class Smart, class... Args>
  out_ptr_t<Smart, POINTER_OF(Smart), Args...> out_ptr(Smart& s, Args&&... args);

// 20.11.11, inout_ptr_t
template <class Smart, class Pointer, class... Args>
  class inout_ptr_t;

// 20.11.12, inout_ptr
template <class Pointer, class Smart, class... Args>
  inout_ptr_t<Smart, Pointer, Args...> inout_ptr(Smart& s, Args&&... args);

template <class Smart, class... Args>
  inout_ptr_t<Smart, POINTER_OF(SMART), Args...> inout_ptr(Smart& s, Args&&... args);
```

Insert §20.11.9 [**out_ptr.class**]:

20.11.9 **Class Template** `out_ptr_t` [**out_ptr.class**]

[1] *out_ptr_t* is a type used with smart pointers ([**smartptr**] 20.11) and types which are designed on the same principles to interoperate easily with functions that use output pointer parameters. [Note — For example, a function of the form `void foo(void**)` — *end note*].

[2] *out_ptr_t* may be specialized ([**temp.class.spec**] 13.6.5) for program-defined types.

```
namespace std {

  template <class Smart, class Pointer, class... Args>
  class out_ptr_t {
  public:
    // 20.11.9.1, constructors
    out_ptr_t(Smart&, Args...);
    out_ptr_t(out_ptr_t&&) noexcept;

    // 20.11.9.2, assignment
    out_ptr_t& operator=(out_ptr_t&&) noexcept;

    // 20.11.9.3, destructors
    ~out_ptr_t() noexcept;

    // 20.11.9.4, conversion operators
    operator Pointer*() noexcept const;
    operator void**() noexcept const;

  private:
    Smart* s; // exposition only
    tuple<Args...> a; // exposition only
    Pointer p; // exposition only
  };

}
```

[2] If `Smart` is a specialization of `shared_ptr` and `sizeof...(Args) == 0`, the program is ill-formed.

[3] [*Note:* It is typically a user error to reset a `shared_ptr` without specifying a deleter, as `shared_ptr` will replace a custom deleter with the default deleter upon usage of `.reset()`, as specified in 20.11.3.4. — *end note*]

20.11.9.1 Constructors [**out_ptr.class.ctor**]

```
out_ptr_t(Smart& smart, Args&&... args);
```

[1] Effects: initializes `s` with `addressof(smart)`, `a` with `std::forward<Args>(args)...`, and initializes `p` to its value initialization.

[2] [*Note:* The constructor is not `noexcept` to allow for a variety of non-terminating and safe implementation strategies. For example, an implementation may allocate a `shared_ptr`'s internal node in the constructor and let implementation-defined exceptions escape safely. The destructor can then move the allocated control block in directly and avoid any other exceptions. — *end note*]

```
out_ptr_t(out_ptr&& rhs) noexcept;
```

[3] Effects: initializes `s` with `std::move(rhs.s)`, `a` with `std::move(args)...`, and `p` with `std::move(rhs.p)`. Then sets `rhs.p` to `nullptr`.

20.11.9.2 Assignment [**out_ptr.class.assign**]

```
out_ptr_t& operator=(out_ptr&& rhs) noexcept;
```

[1] Effects: Equivalent to:

```
s = std::move(rhs.s);
a = std::move(rhs.a);
p = std::move(rhs.p);
rhs.p = nullptr;
return *this;
```

20.11.9.3 Destructors [**out_ptr.class.dtor**]

```
~out_ptr_t();
```

[1] Let SP be POINTER_OF_OR(Smart, Pointer) ([**memory.defns**] 20.10.2).

[2] Effects: Equivalent to:

— if (p) { s->reset(static_cast<SP>(p), std::forward<Args>(args)...); } if the expression s->reset(static_cast<SP>(p), std::forward<Args>(args)...) is well-formed,

— otherwise if (p) { *s = Smart(static_cast<SP>(p), std::forward<Args>(args)...); };

where Args are the arguments stored in a.

20.11.9.4 Conversions [**out_ptr.class.conv**]

```
operator Pointer*() noexcept const;
```

[1] Effects: returns a pointer to p.

```
operator void**() noexcept const;
```

[2] Constraints: is_same_v<remove_cvref_t<Pointer>, void*> is false.

[3] Effects: Equivalent to: returns reinterpret_cast<void**>(static_cast<Pointer*>(*this));.

Insert §20.11.10 [**out_ptr**]:

20.11.10 **Function Template** out_ptr [**out_ptr**]

[1] *out_ptr* is a function template that produces an object of type *out_ptr_t* ([**out_ptr.class**] 20.11.9).

```
template <class Pointer, class Smart, class... Args>
  out_ptr_t<Smart, Pointer, Args...> out_ptr(Smart& s, Args&&... args);
```

[2] Returns: return out_ptr_t<Smart, Pointer, Args...>(s, std::forward<Args>(args)...);

```
template <class Pointer, class Smart, class... Args>
  out_ptr_t<Smart, Pointer, Args...> out_ptr(Smart& s, Args&&... args);
```

[3] Returns: return out_ptr_t<Smart, POINTER_OF(Smart), Args...>(s, std::forward<Args>(args)...);

Insert §20.11.11 [**inout_ptr.class**]:

20.11.11 **Class Template** `inout_ptr_t` [**inout_ptr.class**]

[1] *inout_ptr_t* is a type used with smart pointers ([**smartptr**] 20.11) and types which are designed on the same principles to interoperate easily with functions that use output pointer parameters. [*Note:* For example, a function of the form `errno_t foo_realloc(void**)` — *end note*].

[2] *inout_ptr_t* may be specialized ([**temp.class.spec**] 13.6.5) for program-defined types.

```
namespace std {

  template <class Smart, class Pointer, class... Args>
  class inout_ptr_t {
  public:
    // 20.11.11.1, constructors
    inout_ptr_t(Smart&, Args...);
    inout_ptr_t(inout_ptr_t&&) noexcept;

    // 20.11.11.2, assignment
    inout_ptr_t& operator=(inout_ptr_t&&) noexcept;

    // 20.11.11.3, destructors
    ~inout_ptr_t();

    // 20.11.11.4, conversion operators
    operator Pointer*() noexcept const;
    operator void**() noexcept const;

  private:
    Smart* s; // exposition only
    tuple<Args...> a; // exposition only
    Pointer p; // exposition only
  };

}
```

[2] If `Smart` is a specialization of `shared_ptr`, the program is ill-formed.

[3] [*Note:* It is impossible to properly release the managed resource from a `shared_ptr` given its shared ownership model. — *end note*]

20.11.11.1 Constructors [**inout_ptr.class.ctor**]

```
inout_ptr_t(Smart& smart, Args... args);
```

[1] Effects: initializes `s` with `addressof(smart)`, `a` with `std::forward<Args>(args)...`, and `p` to either

— `smart` if `std::is_pointer_v<Smart>` is true,

— otherwise, an unspecified value of either `smart.get()` or its value initialization.

[2] [*Note:* An unspecified value allows an implementation and subsequent program-defined specializations to pick an option which fits an implementation's purpose and potential implementation-specific optimizations. — *end note*].

[3] Remarks: if an implementation calls `smart.release()` or `s->release()`, then it shall not call `s->release()` in the destructor.

[4] [*Note:* The constructor is not `noexcept` to allow for a variety of non-terminating and safe implementation strategies. For example, an intrusive pointer with a control block may allocate in the constructor and safely fail with an exception. — *end note*]

```
inout_ptr_t(inout_ptr&& rhs) noexcept;
```

[4] Effects: initializes `s` with `std::move(rhs.s)`, `a` with `std::move(args)...`, and `p` with `std::move(rhs.p)`. Then sets `rhs.p` to `nullptr`.

20.11.11.2 Assignment [**inout_ptr.class.assign**]

```
inout_ptr_t& operator=(inout_ptr&& rhs) noexcept;
```

[1] Effects: Equivalent to:

```
s = std::move(rhs.s);
a = std::move(rhs.a);
p = std::move(rhs.p);
rhs.p = nullptr;
return *this;
```

20.11.11.3 Destructors [**inout_ptr.class.dtor**]

```
~inout_ptr_t();
```

[1] Constraints: Either `std::is_pointer_v<Smart>` is `true` or the expression `s->release()` is well-formed.

[2] Let `SP` be `POINTER_OF_OR(Smart, Pointer)` ([**memory.defns**] 20.10.2).

[3] Effects: Equivalent to:

— `if (p) { *s = Smart(static_cast<SP>(p), std::forward<Args>(args)...); }` if `std::is_pointer_v<Smart>` is true,

— `if (p) { s->release(); s->reset(static_cast<SP>(p), std::forward<Args>(args)...); }` if the expression `s->reset(static_cast<SP>(p), std::forward<Args>(args)...)` is well-formed,

— otherwise, `if (p) { s->release(); *s = Smart(static_cast<SP>(p), std::forward<Args>(args)...); }`.

where `Args` are the arguments stored in `a`.

20.11.11.4 Conversions [**inout_ptr.class.conv**]

```
operator Pointer*() noexcept const;
```

[1] Effects: returns a pointer to `p`.

```
operator void**() noexcept const;
```

[2] Constraints: `is_same_v<remove_cvref_t<Pointer>, void*>` is false.

[3] Effects: Equivalent to: `returns reinterpret_cast<void**>(static_cast<Pointer*>(*this));`.

Insert §20.11.12 [**inout_ptr**]:

20.11.12 **Function Template** `inout_ptr` [**inout_ptr**]

[1] *inout_ptr* is a function template that produces an object of type *inout_ptr_t* ([**inout_ptr.class**] 20.11.11).

```
template <class Pointer, class Smart, class... Args>
  inout_ptr_t<Smart, Pointer, Args...> inout_ptr(Smart& s, Args&&... args);
```

[2] Returns: `inout_ptr_t<Smart, Pointer, Args...>(s, std::forward<Args>(args)...);`

```
template <class Smart, class... Args>
  inout_ptr<Smart, POINTER_OF(Smart), Args...> inout_ptr(Smart& s, Args&&... args);
```

[3] Returns: `inout_ptr_t<Smart, POINTER_OF(Smart), Args...>(s, std::forward<Args>(args)...);`

§ 8. Acknowledgements

Thank you to Lounge<C++>'s Cicada, melak47, rmf, and Puppy for reporting their initial experiences with such an abstraction nearly 5 years ago and helping JeanHeyd Meneide implement the first version of this.

Thank you to Mark Zeren for help in this investigation and analysis of the performance of smart pointers.

Thank you to Tim Song for reviewing the wording for this paper and vastly improving it!

Thank you to Zach Laine and the Boost Community for reviewing the code.

§ References

§ Informative References

[ADOBE-OUT-PTR]
Adobe. Adobe Chromium: scoped_comptr. November 25th, 2018. URL: https://github.com/adobe/chromium/blob/master/base/win/scoped_comptr.h#L80

[CCOMPTR]
Microsoft. CComPtr::operator& Operator. 2015. URL: https://msdn.microsoft.com/en-us/library/31k6d0k7.aspx

[N4820]
Richard Smith. Working Draft, Standard for Programming Language C++. 18 June 2019. URL: https://wg21.link/n4820

[OLDNEWTHING-HANDLE-PROXY]
Raymond Chen. Spotting problems with destructors for C++ temporaries. April 29th, 2019. URL: https://devblogs.microsoft.com/oldnewthing/20190429-00/?p=102456

[P0468]
Isabella Muerte. A Proposal to Add an Intrusive Smart Pointer to the C++ Standard Library. October 15th, 2016. URL: http://wg21.link/p0468

[STD-PROPOSALS-OVERLOAD-OPERATOR]
isocpp.org Forums. Add operator&() to std::unique_ptr to get internal pointer. April 15th, 2018. URL: https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/8MQhnL9rXBI

[WRL-COMPTRREF]
Microsoft. ComPtrRef Class. November 4th, 2016. URL: https://docs.microsoft.com/en-us/cpp/windows/comptrref-class